

AI Search Framework: Sample Pipeline Execution & Response

End-to-End Execution Trace & Judge Verification (Query V2_SD_CODE_04)

Query ID: V2_SD_CODE_04 | **Architecture:** Zero-LLM Proposed Pipeline ($\leq 8B$) | **Specialist:** Qwen-2.5-Coder-7B-Instruct | **Judge:** Qwen-2.5-27B (Groq API, Blind Pairwise)

1. Input Prompt & Execution Trace

User Query:
"Implement an advanced algorithmic module #4 in Python with full type annotations, $O(1)/O(\log N)$ complexity guarantees, and comprehensive edge-case unit tests."

Stage	Component	Model / Algorithm	Params	Latency	Action & Output Detail
1. Decomposition	Decomposition SLM	Qwen/Qwen2.5-7B-Instruct	7.61B	12.4s	Evaluated query; emitted atomic 1-node DAG (node_1) under Fix 2 stop-condition.
2. Routing	Task Colourer	Centroid Cosine Similarity	0B	0.02s	Classified domain: coding=0.912 , color: blue . Dispatched to Code Specialist.
3. Specialist Execution	Code Specialist SLM	Qwen/Qwen2.5-Coder-7B-Instruct	7.61B	114.1s	Generated typed MinMaxHeap supporting $O(1)$ min/max and $O(\log N)$ insert/delete.
4. Aggregation	Two-Stage Aggregator	meta-llama/Llama-3.1-8B-Instruct	8.03B	0.0s	Single-subtask bypass (Fix 1). Pristine specialist code returned with zero compression.
Total System	All-SLM Pipeline	All models $\leq 8B$	Max 8.03B	126.6s	4,206 chars returned (100% code integrity preserved)

2. Complete Verbatim Response from Proposed SLM Pipeline

```
"""
Advanced Algorithmic Module #4: Efficient Data Structure Operations

This module implements several advanced algorithmic operations with:
- Full type annotations
-  $O(1)$  or  $O(\log N)$  complexity guarantees
- Comprehensive edge-case unit tests
"""

from __future__ import annotations

import math
from typing import Any, Dict, List, Optional, Tuple, Union

class MinMaxHeap:
    """
    A min-max heap that supports  $O(1)$  access to both minimum and maximum elements,
    and  $O(\log N)$  insertion and deletion.

    The min-max heap is a complete binary tree where nodes at even levels (0-indexed)
    are min-nodes and nodes at odd levels are max-nodes.
    """

    def __init__(self) -> None:
        self._heap: List[int] = []

    def __len__(self) -> int:
        return len(self._heap)

    def __bool__(self) -> bool:
        return len(self._heap) > 0

    def _parent(self, i: int) -> int:
        return (i - 1) // 2

    def _left_child(self, i: int) -> int:
        return 2 * i + 1

    def _right_child(self, i: int) -> int:
        return 2 * i + 2

    def _grandparent(self, i: int) -> int:
        return self._parent(self._parent(i))

    def _is_min_level(self, i: int) -> bool:
        """Check if node at index i is at a min-level (even level)."""
        level = self._level(i)
        return level % 2 == 0

    def _level(self, i: int) -> int:
        """Calculate the level of node at index i."""
        return int(math.log2(i + 1)) if i >= 0 else 0

    def _bubble_up(self, i: int) -> None:
        """Bubble up the element at index i to restore min-max heap property."""
        while i > 0:
            if self._is_min_level(i):
                if self._heap[i] < self._heap[self._parent(i)]:
                    self._heap[i], self._heap[self._parent(i)] = (
                        self._heap[self._parent(i)],
                        self._heap[i],
                    )
                i = self._parent(i)
            if i > 0 and self._heap[i] > self._heap[self._grandparent(i)]:
                self._heap[i], self._heap[self._grandparent(i)] = (
```

```

        self._heap[self._grandparent(i)],
        self._heap[i],
    )
    i = self._grandparent(i)
else:
    break
else:
    if self._heap[i] > self._heap[self._parent(i)]:
        self._heap[i], self._heap[self._parent(i)] = (
            self._heap[self._parent(i)],
            self._heap[i],
        )
    i = self._parent(i)
    if i > 0 and self._heap[i] < self._heap[self._grandparent(i)]:
        self._heap[i], self._heap[self._grandparent(i)] = (
            self._heap[self._grandparent(i)],
            self._heap[i],
        )
    i = self._grandparent(i)
else:
    break

def push(self, val: int) -> None:
    """Insert val into the heap. O(log N) time."""
    self._heap.append(val)
    self._bubble_up(len(self._heap) - 1)

def get_min(self) -> int:
    """Return the minimum element. O(1) time."""
    if not self._heap:
        raise IndexError("get_min from empty heap")
    return self._heap[0]

def get_max(self) -> int:
    """Return the maximum element. O(1) time."""
    if not self._heap:
        raise IndexError("get_max from empty heap")
    if len(self._heap) == 1:
        return self._heap[0]
    if len(self._heap) == 2:
        return self._heap[1]
    return max(self._heap[1], self._heap[2])

```

3. Independent Blind Judge Scorecards for Query V2_SD_CODE_04

Baseline Competitor	Baseline Scale	Forward Trial Verdict	Swapped Trial Verdict	Overall Query Outcome
Gemini-1.5-Pro	Frontier Commercial API	SLM Wins (Score: 3/2/3 vs 1/1/2)	SLM Wins (Score: 3/2/3 vs 1/1/2)	100% Win (2 / 2)
Llama-3.1-70B-Instruct	70B Monolithic Open Model	SLM Wins (Score: 4/3/4 vs 2/2/3)	SLM Wins (Score: 4/3/4 vs 2/2/3)	100% Win (2 / 2)
Llama-3.1-8B-Instruct	8B Monolithic Baseline	SLM Wins (Score: 3/2/3 vs 1/1/2)	SLM Wins (Score: 3/2/3 vs 1/1/2)	100% Win (2 / 2)
Qwen-2.5-32B-Instruct	32B Dense Open Model	Baseline Wins	SLM Wins	50% Win (1 / 2)
Qwen-2.5-72B-Instruct	72B Monolithic Open Model	Baseline Wins	SLM Wins	50% Win (1 / 2)
Total Performance on Query	All 5 Baselines Combined	3 Wins / 2 Losses	5 Wins / 0 Losses	80.0% Win Rate (8 Wins / 2 Losses)